

Plataforma .NET

Caso de estudio

Arquitectura del Software

Curso 2025/26

Introducción

- En esta presentación se desarrolla la implementación del servicio REST *Bookle* en la plataforma .NET y con persistencia MongoDB. `BookleApi` es un API basada en controlador, mientras que `BookleMinApi` es un API mínima.
- Incluye fragmentos de código que ilustran aspectos destacados del lenguaje C# y la plataforma.
- El código completo del proyecto está disponible en el repositorio de la asignatura.

Creación del proyecto. API basada en controlador

Desde el terminal, creamos un proyecto *API Web*:

```
dotnet new webapi -o BookleApi  
cd BookleApi  
code -r ../BookleApi
```

Dentro de la carpeta del proyecto, añadimos las dependencias:

```
dotnet add package MongoDB.Driver
```

- Repositorio de paquetes .NET: <https://www.nuget.org/packages>
- El fichero `BookleApi.csproj` contiene la configuración del proyecto (similar al `pom.xml` de Maven).

Organización del código del proyecto

La organización del código en C# es flexible. Por simplicidad en los proyectos, el código se organiza en las siguientes carpetas:

- `Models` : contiene el código fuente del modelo de dominio y repositorios.
- `Repositories` : contiene el código de los repositorios.
- `Services` : código de los servicios.
- `Controllers` : implementación de los controladores REST (proyecto API basado en controlador).
- `Endpoints` : código del API (proyecto API mínima)

Los ficheros de código fuente tienen extensión `.cs` . Podemos tener varios tipos de datos en un solo fichero.

Organización del código del proyecto

En un fichero de código fuente, los tipos de datos se declaran dentro de un **espacio de nombres** (fichero `BookleModelo.cs`):

```
namespace BookleApi.Models
{
    public class Actividad
    {
        // ...
    }
    // ...
}
```

Organización del código del proyecto

Hacemos uso de los tipos de datos de un espacio de nombres con la declaración `using` (fichero `BookleServicio.cs`):

```
using BookleApi.Models;  
using BookleApi.Repositories;  
using BookleApi.DTOS;  
  
namespace BookleApi.Services;  
    // ...
```

Modelo

El *agregado* `Actividad` tiene las anotaciones (atributos en .NET) de MongoDB.

```
public class Actividad
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string? Id { get; set; } //= ObjectId.GenerateNewId().ToString();
    public string Titulo { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public string Profesor { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public List<DiaActividad> Agenda { get; set; } = [];
}
```

- Inicializamos las propiedades que no deben ser nulas para evitar los avisos de `nullable reference types`.

Modelo

Aspectos a destacar del código anterior:

- El lenguaje C# incluye el concepto de **propiedad** de una clase.
 - En la declaración de una propiedad indicamos si se permite la lectura (`get`) y la modificación (`set`).
 - Las propiedades se pueden inicializar en su declaración (`Agenda`).
 - Ejemplo de uso: `actividad1.Titulo = "Entrevistas";`
- La convención de nombrado en C# indica que métodos y propiedades se nombran con la primera letra en mayúscula.
- Una propiedad calculada se declararía como:
`public int DiasAgenda => Agenda.Count;`

Repositorio

La interfaz que define el patrón *Repositorio* define los métodos asíncronos. El driver MongoDB para .NET está pensado para trabajar con operaciones asíncronas y es lo recomendado.

```
public interface IRepository<T, K>    {  
    Task<K> AddAsync(T entity);  
    Task UpdateAsync(T entity);  
    Task DeleteAsync(T entity);  
    Task<T?> GetByIdAsync(K id);  
    Task<List<T>> GetAllAsync();  
    Task<List<K>> GetIdsAsync();  
}
```

Aunque cambiáramos a un repositorio *in memory*, podemos mantener la implementación asíncrona por consistencia.

Repositorio

Inyectamos el `MongoClient` (para que solo se cree una vez) y la configuración de la que obtener la cadena de conexión:

```
public class RepositorioActividadesMongoDB : IRepository<Actividad, string>
{
    private readonly IMongoCollection<Actividad> _actividades;
    public RepositorioActividadesMongoDB(IMongoClient client, IConfiguration config)
    {
        var database = client.GetDatabase(config["MongoDB:Database"]);
        _actividades = database.GetCollection<Actividad>(
            config["MongoDB:Collection"] ?? "actividades.net");
    }
    // ...
}
```

Repositorio

Del código anterior destacamos:

- El modificador `readonly` indica que una variable es de solo lectura (`final` de Java).
- El guion bajo es convención de nombres. Indica que es un campo privado.
- La herencia de clases y la implementación de interfaces se expresan en C# del mismo modo: utilizando `:`
- El modificador `var` en una variable indica al compilador que deduzca su tipo a partir del objeto que es asignado.

Repositorio

- En `Program.cs` inyectamos como singleton el `MongoClient` para que esté disponible en el contenedor de dependencias. `IConfiguration` está disponible automáticamente.

```
builder.Services.AddSingleton<IMongoClient>(
    new MongoClient(builder.Configuration["MongoDB:Uri"]));
```

- La configuración se define en `appsettings.json`

```
{
  "MongoDB": {
    "Uri": "mongodb://localhost:27017",
    "Database": "arso2026",
    "Collection": "actividades.net"
  }
}
```

Repositorio

Inserción en el repositorio:

```
public async Task<string> AddAsync(Actividad entity)
{
    await _actividades.InsertOneAsync(entity);
    return entity.Id;
}
```

- Las operaciones del *driver* de MongoDB notifican excepciones. Sin embargo, no es obligatorio declararlas o atraparlas.
- Podemos obtener del objeto el identificador tras la inserción.

Repositorio

Actualización de una entidad

```
public async Task UpdateAsync(Actividad entity)
{
    await _actividades.ReplaceOneAsync(a => a.Id == entity.Id, entity);
}
```

- El primer parámetro del método `ReplaceOne` es una **expresión lambda** que establece el selector.

Repositorio

Obtención de los identificadores del repositorio:

```
public async Task<List<string>> GetIdsAsync()
{
    var todas = await _actividades.Find(_ => true).ToListAsync();
    return todas.Select(a => a.Id).ToList();
}
```

- En la expresión lambda `_ => true` se utiliza `_` para omitir el nombre del parámetro, ya que no es relevante para la evaluación.
- El método `Select` realiza la proyección de un campo del objeto `Actividad`.
- `IMongoCollection` no tiene `Select`, hacemos `Find` para obtener un cursor.

Servicio

- El servicio define método asíncronos también.

```
public interface IServicioBookle
{
    Task<string> CreateAsync(Actividad actividad);
    Task<Resultado> UpdateAsync(Actividad actividad);
    Task<Resultado> RemoveAsync(string id);
    Task<Resultado> ReservarAsync(string idActividad,
        DateOnly fecha, int indice, string alumno, string email);
    Task<Actividad?> GetAsync(string id);
    Task<List<ActividadResumen>> GetResumenesAsync();
}
```

Servicio

- Para hacer explícitos los errores en las acciones, creamos un *record*

Resultado .

```
public record Resultado(bool Success, string? Message)
{
    public static Resultado Ok() => new(true, null);
    public static Resultado Error(string msg) => new(false, msg);
    public static Resultado NotFound(string msg) => new(false, msg);
    public static Resultado Conflict(string msg) => new(false, msg);
}
```

- Además, el DTO `ActividadResumen.cs` también se define como un *record*.

```
public record ActividadResumen(string Id, string Titulo, string Profesor);
```

Servicio

Implementación del servicio utilizando el repositorio:

```
public class ServicioBookle : IServicioBookle
{
    private readonly IRepository<Actividad, string> _repositorio;

    public ServicioBookle(IRepository<Actividad, string> repositorio) {
        _repositorio = repositorio;
    }
    // ...
}
```

- El repositorio de actividades (interfaz) se establece en el constructor utilizando un mecanismo de **inyección de dependencias**.

Servicio

Las operaciones del servicio pueden realizar **control de precondiciones**:

```
public async Task<Resultado> ReservarAsync(String id,  
DateOnly fecha, int indice, String alumno, String email)  
{  
    if (indice < 1)  
        return Resultado.Error("Los índices comienzan en 1");  
    //...
```

Inyección de dependencias

En la clase `Program` se realiza la configuración de repositorios y servicios.

```
builder.Services.AddScoped<IRepositorio<Actividad, String>, RepositorioActividadesMongoDB>();  
builder.Services.AddScoped<IServicioBookle, ServicioBookle>();
```

- Al añadir `MongoClient` al contenedor de dependencias usamos `AddSingleton`. Este método genérico registra una instancia de una clase.
- Con el repositorio y el servicio usamos `AddScoped`, que crea una instancia por cada petición HTTP.
- El sistema de inyección de dependencias establece la instancia en aquellos servicios que indican la dependencia como un parámetro en el constructor.

Controlador REST

```
[Route("api/actividades")]
[ApiController]
public class ActividadesController : ControllerBase
{
    private readonly IServicioBookle _servicio;

    public ActividadesController(IServicioBookle servicio)
    {
        _servicio = servicio;
    }
    // ...
}
```

- Obtiene la instancia del servicio gracias a la inyección de dependencias.
- La anotación `Route` establece el path de acceso al servicio.

Controlador REST

- En `Program.cs` añadimos:

```
builder.Services.AddControllers();
```

- Para que los controladores se queden registrados.
- Y en la fase del pipeline añadimos:

```
app.MapControllers();
```

- Para que se conecten las rutas de los controllers al sistema de routing y estén activas.

Controlador REST

Método de recuperación:

```
[HttpGet("{id}", Name = "GetActividad")]  
public async Task<ActionResult<Actividad>> Get(string id)  
{  
    var actividad = await _servicio.GetAsync(id);  
  
    if (actividad == null)  
    {  
        return NotFound();  
    }  
    return Ok(actividad);  
}
```

Controlador REST

Sobre el método anterior:

- `ActionResult` corresponde con la respuesta HTTP. Se parametriza con la clase que representa el documento de respuesta.
- La llamada al método `NotFound()` retorna una respuesta "not found" de HTTP.
- `Ok(actividad)` devuelve respuesta HTTP 200 y el objeto actividad convertido a JSON.
- La propiedad `Name = "GetActividad"` de la anotación `HttpGet` le asigna un nombre lógico a la operación para redirigir la operación de creación.

Controlador REST

```
[HttpPost]
public async Task<ActionResult<Actividad>> Create(Actividad actividad)
{
    var id = await _servicio.CreateAsync(actividad);
    return CreatedAtRoute("GetActividad", new { id }, actividad);
}
```

- En ASP.NET Core es convención que las operaciones de creación retornen el recurso.
- El método `CreatedAtRoute` retorna la respuesta HTTP "creado" y la cabecera Location con la URL de la nueva entidad.

Controlador REST

```
[HttpPost("{id}/agenda/{fecha}/turno/{indice}/reserva")]
public async Task<IActionResult> Reservar(string id,
    DateOnly fecha, int indice, [FromForm] string alumno,
    [FromForm] string email)
{
    var result = await _servicio.ReservarAsync(id, fecha, indice, alumno, email);

    if (!result.Success)
        return BadRequest(result.Message);
    return NoContent();
}
```

- Los parámetros de tipo *path* y *body* no es necesario que tengan anotaciones. En cambio, los parámetros de tipo *form* se especifican con `[FromForm]`. Los de tipo *query* se indican con `[FromQuery]`.
- Cuando una operación no retorna contenido declara `IActionResult`.

Tratamiento de errores

- Creamos un middleware de excepciones:

```
public class ExceptionMiddleware{
    private readonly RequestDelegate _next;

    public ExceptionMiddleware(RequestDelegate next)    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (ArgumentException ex)
        {
            //..
        }
        catch (Exception)
        {
            //..
        }
    }
}
```

Tratamiento de errores

En `Program.cs` tiene que ser la primera línea del pipeline, antes de cualquier otro middleware, para que capture todas las excepciones

```
app.UseMiddleware<ExceptionMiddleware>();  
app.MapControllers();  
app.Run();
```

Ejecución

- Ejecutamos la aplicación pulsando `CTRL+F5` .
- La aplicación atiende peticiones HTTPS y HTTP en puertos diferentes.
- En Windows y MacOS debemos instalar un certificado:

```
dotnet dev-certs https --trust
```

- La herramienta Swagger UI está accesible en `.../swagger/`. Se configuró al crear la plantilla con `dotnet new webapi`

Creación del proyecto. API Mínima

Desde el terminal, creamos un proyecto *API Web*:

```
dotnet new web -o BookleMinApi  
cd BookleMinApi  
code -r ../BookleMinApi
```

Igual que en el anterior proyecto, dentro de la carpeta del proyecto, añadimos las dependencias:

```
dotnet add package MongoDB.Driver
```

Código generado

- Como resultado, se ha creado un proyecto básico. `Program.cs` :

```
var builder = WebApplication.CreateBuilder(args);  
var app = builder.Build();  
app.MapGet("/", () => "Hello World!");  
app.Run();
```

- Se crean los elementos `WebApplicationBuilder` y `WebApplication` con valores predeterminados preconfigurados.
- Se crea un punto de conexión HTTP GET / que devuelve Hello World!.
- `app` tiene los métodos `MapGet` , `MapPost` , ..., `Map<HttpVerb>` . Podríamos crear todos los puntos de conexión en `Program.cs` , pero vamos a organizar mejor el código.

Modelo, repositorio y servicio

- El código del dominio, el repositorio y el servicio coincide con el proyecto anterior.
- Ahora ya no tenemos una clase controlador que herede de `ControllerBase`.
- Para configurar los puntos de conexión vamos a hacer uso de `MapGroup` para organizar todos los que comparten un prefijo común.
- Además, vamos a hacer que los métodos `Map<HttpVerb>` invoquen métodos en lugar de usar expresiones lambda.

Endpoint

- `ActividadesEndpoints` es una clase estática en `Endpoints/` que define un `MapGroup` con el prefijo de todas las llamadas.
- `MapActividadesEndpoints(this WebApplication app)` extiende los métodos de `WebApplication`.

```
public static class ActividadesEndpoints
{
    public static void MapActividadesEndpoints(this WebApplication app)    {
        var group = app.MapGroup("/api/actividades");
        group.MapGet("/", Get);
        group.MapGet("/{id}", GetById).WithName("GetActividad");
        group.MapPost("/", Create);
        group.MapPut("/{id}", Update);
        group.MapDelete("/{id}", Delete);
        group.MapPost("/{id}/agenda/{fecha}/turno/{indice}/reserva", Reservar);
    }
}
```

Obtener resúmenes de Actividad

```
private static async Task<IResult> Get(IServicioBookle servicio)
{
    var result = await servicio.GetResumenesAsync();
    return Results.Ok(result);
}
```

- `IResult` es una interfaz que representa una respuesta HTTP en Minimal APIs.
- `Results` es una clase estática helper. Sirve para crear respuestas HTTP fácilmente.
- Este método está devolviendo un objeto que representa una respuesta HTTP 200 y los datos en formato JSON.

Crear actividad

```
private static async Task<IResult> Create(Actividad actividad,
IServicioBookle servicio)
{
    var id = await servicio.CreateAsync(actividad);

    return Results.CreatedAtRoute("GetActividad", new { id }, actividad);
}
```

- El servicio se inyecta en cada uno de los métodos.
- Podemos darle nombre al *endpoint* que hace un *get by id* en el `MapGroup` e invocarlo en el crear igual que hacíamos en la versión con controlador.

Reservar actividad

- En este caso los datos de la reserva llegan como un JSON.

```
public record ReservaRequest(string Alumno, string Email);  
//  
static async Task<IResult> Reservar(  
    string id, DateOnly fecha, int indice, ReservaRequest datosReserva,  
    IServicioBookle servicio)  
{  
    var result = await servicio.ReservarAsync(id, fecha, indice,  
        datosReserva.Alumno, datosReserva.Email);  
  
    if (!result.Success)  
        return Results.BadRequest(result.Message);  
  
    return Results.NoContent();  
}
```

Configuración de Program.cs

- Añadimos igual que antes las instancias que queremos disponibles para inyección de dependencias:

```
builder.Services.AddSingleton<IMongoClient>(  
    new MongoClient(builder.Configuration["MongoDB:Uri"]));  
builder.Services.AddScoped<IRepositorio<Actividad, String>, RepositorioActividadesMongoDB>();  
builder.Services.AddScoped<IServicioBookle, ServicioBookle>();
```

- Configuramos las rutas de conexión de Mongo en `appsettings.json`.
- Y en `Program.cs` añadimos al pipeline nuestro endpoint:

```
app.MapActividadesEndpoints();  
app.Run();
```

Creación de una interfaz de pruebas de API con Swagger

- Ejecutamos en la terminal:

```
dotnet add package NSwag.AspNetCore
```

- En Program.cs añadimos antes de definir `app` :

```
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddOpenApiDocument(config =>  
{  
    config.DocumentName = "BookleMinApi";  
    config.Title = "BookleMinApi v1";  
    config.Version = "v1";  
});
```

Creación de una interfaz de pruebas de API con Swagger

- Después de definir `app` añadimos:

```
if (app.Environment.IsDevelopment())
{
    app.UseOpenApi();
    app.UseSwaggerUi(config =>
    {
        config.DocumentTitle = "BookleMinApi";
        config.Path = "/swagger";
        config.DocumentPath = "/swagger/{documentName}/swagger.json";
        config.DocExpansion = "list";
    });
}
```

- La herramienta estará accesible en `.../swagger`.

Ejecución

- Igual que antes, ejecutamos la aplicación pulsando `CTRL+F5` .
- También podemos añadir el middleware para el tratamiento de errores.
- La aplicación atiende peticiones HTTPS y HTTP en puertos diferentes.
- En Windows y MacOS debemos instalar un certificado:

```
dotnet dev-certs https --trust
```